claude-windows / 8d4866bf-8edd-4da9-909e-022ddee12675

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\8d4866bf-8edd-4da9-909e-022ddee12675\subagents\workflows\wf_f9255361-69d\agent-a4cdcd8b3fa114424.jsonl`
- SHA-256: `16c86a0017c1539982a7122950e2fed4c45c865dc2c3ab8e8ad1636affd6c377`
- Source modified: `2026-06-22T09:35:52+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_f9255361-69d`
- session_id: `8d4866bf-8edd-4da9-909e-022ddee12675`

Transcript

1. user (2026-06-22T09:34:22.825Z)

You are auditing ONE doc in the KhelSutra brand/site repo for staleness, as part of a "are all the docs up to date?" sweep.

Repo root: C:\Users\avidu\Projects\khelsutra (bash path: /c/Users/avidu/Projects/khelsutra)
The repo is on `master`, freshly synced to remote HEAD ? the latest merged PR is #94. Today is 2026-06-22.

DOC TO AUDIT: web/README.md

Your job: read the doc IN FULL, extract every concrete, falsifiable claim, and VERIFY each against the LIVE repo ? do NOT trust the doc's own wording. Hard discipline rule: a claim is "correct" ONLY when you have file:line or command-output evidence for it; if you cannot verify it, mark verdict "unverifiable" ? never guess, never assume the doc is right. Re-check against actual code/git.

Kinds of claims to check:
- File / path / route references ("served at /capture", "see src/store.js", "public/og.png", "systemd khelsutra-site") -> does the path or route ACTUALLY exist now? Confirm with git ls-files / Read / the routing in site/server.js + site/src/index.js.
- PR numbers and counts ("PRs #1-#8 merged", "20 tests green", "6 languages", "3-step wizard", "P0..P5 done") -> verify against `git -C <root> log`, the actual test output/count, and the code.
- Status markers (DONE / pending / IN PROGRESS / DRAFT / "?" / "owner-gated" / "NEXT") -> still accurate vs the code & git history? Is something marked pending that is actually shipped, or vice-versa?
- Dates, "live at <url>", "merged in #N".
- Whether the doc describes something now SUPERSEDED or fully shipped -> is it an archive candidate (a tracker whose work is complete, or content duplicated by docs/archives/)? Note: docs/archives/FRIENDLY_AND_MULTILINGUAL.md already exists as the archived multilingual tracker.

Tools: Read the doc + every cited file; Grep/Glob to locate code; Bash for `git -C /c/Users/avidu/Projects/khelsutra log --oneline`, `... ls-files`, and greps. To check a test count, you may run the suite (`cd /c/Users/avidu/Projects/khelsutra/site && node --test` or `npm test`), but prefer cheap checks. Verify against master as it stands NOW.

Severity: high = a factually wrong claim a reader would act on (wrong path/route, a "DONE" that isn't, a wrong count/PR that misleads). medium = outdated-but-harmless (stale PR range, old "where we are", superseded framing). low = cosmetic.

Do NOT modify any file ? audit only. Return ONLY the structured object (doc, overallStatus, findings[], summary).

2. assistant (2026-06-22T09:34:25.255Z)

I'll audit `web/README.md` in the KhelSutra repo. Let me start by reading the doc and syncing the repo state.

3. user (2026-06-22T09:34:26.302Z)

```
1       # web/ ? KhelSutra Studio (operator console SPA)
2
3       Vite + React + TS + Tailwind v4. The browser UI that **drives the rally indexer
engine**:
4       upload ? index ? pick rallies ? reel ? download. See `docs/PER_RALLY_SELECTION.md` (the
5       per-rally curation screen) and `docs/LOCAL_APPLIANCE_ARCHITECTURE.md` (the single-box
appliance
6       this SPA fronts).
7
8       ## Dev
9       ```
10      npm install
11      npm run dev        # http://localhost:5173 ; proxies /api ? http://localhost:8000 (the
engine)
12      ```
13      Run the engine alongside it: `python -m backend.main --server` (in the badminton-
highlight-indexer
14      repo). The SPA uses **same-origin** relative `/api/*` calls ? in the appliance a reverse
proxy serves
15      this bundle and proxies `/api` to the engine on `127.0.0.1:8000`.
16
17      ## Checks (CI gates merge)
18      ```
19      npm run typecheck  # tsc --noEmit
20      npm run test       # vitest run  (unit tests; e.g. the typed API client)
21      npm run build      # vite build
22      ```
23
24      ## Layout
25      - `src/api/client.ts` ? typed engine client (`listRallies` / `compileReel` /
`highlightUrl` / `getConfig`),
26        grounded in the verified engine contract; every call carries a correlation id.
27      - `src/api/types.ts` ? engine data types (honest: `server`/`receiver`/`events` are
motion-only
28        fabrications and must not be surfaced).
29      - `src/lib/log.ts` ? structured client logging + correlation ids (observability seed,
appliance §5).
30      - `src/App.tsx` ? the console shell (screens land in M2+).
31
```

4. user (2026-06-22T09:34:27.575Z)

```
50b2bd7 chore(repo): ignore local runtime artifacts + correct stale NEXT_STEPS claims (#94)
e2570e4 docs: archive the Friendly & Multilingual tracker (engineering complete) (#93)
0bef61e docs(friendly-multilingual): close out the chapter ? log #89/#90/#91 + strengthen fitNav
tests (#92)
9f77ddc test(site): browser layout-regression CI guard + auto-collapse the nav to fit every
locale (#91)
4415fe8 feat(site): localize remaining home sections (6 langs) + smooth the responsive header
(#90)
26c9da1 feat(site): localize the hero demo mockup + #how feature block (6 languages) (#89)
7ae9cee feat(web): P2c first-run setup wizard (3 steps from /api/capabilities, 6 languages)
(#88)
cf59fbf feat(site): add es/da/id to the marketing site (parity with the SPA's 6 languages) (#87)
17242a9 feat(site): localize own-model body content to kn/hi (#86)
7b04f02 deploy(kit): align with the live-verified droplet deploy (+ the gemini-default fix)
(#85)
5e5a2ac docs(friendly-multilingual): close out the chapter (§9 DoD + UI/UX + B7 deferred) (#84)
4c0f35c fix(site): UI/UX ? compact language switcher + header overflow fix + localized hero
(#83)
```

```
0fae364 feat(web+site): A2 ? localized WhatsApp share for the reel (SPA + site) (#82)
1e52cfd deploy(droplet): tailored alpha kit for the CPU droplet (engine api. side) (#81)
64076a5 test(web): add a coverage floor (CI-gated) + i18n regression tests for the #77 audit
(#80)
---HEAD---
50b2bd75f9178d54de6a65887b08b7dae29fad56
---BRANCH---
master
```

5. assistant (2026-06-22T09:34:30.101Z)

```
The doc references files and other docs. Let me verify the cited paths and routes against the
live repo.
```

6. user (2026-06-22T09:34:33.436Z)

```
=== web/ tracked files ===
web/.gitignore
web/README.md
web/index.html
web/package-lock.json
web/package.json
web/src/App.tsx
web/src/api/client.test.ts
web/src/api/client.ts
web/src/api/locale.test.ts
web/src/api/types.ts
web/src/api/upload.test.ts
web/src/components/CompileResult.tsx
web/src/components/IngestPanel.test.tsx
web/src/components/IngestPanel.tsx
web/src/components/LanguageSwitcher.tsx
web/src/components/QueryBar.examples.test.ts
web/src/components/QueryBar.test.tsx
web/src/components/QueryBar.tsx
web/src/components/RallyGrid.tsx
web/src/components/SelectionFooter.test.tsx
web/src/components/SelectionFooter.tsx
web/src/components/SetupWizard.test.tsx
web/src/components/SetupWizard.tsx
web/src/components/StatusChip.tsx
web/src/components/TapFilters.test.tsx
web/src/components/TapFilters.tsx
web/src/components/a4.touch.test.tsx
web/src/components/a8.status.test.tsx
web/src/components/build-error.test.tsx
web/src/components/upload.test.tsx
web/src/hooks/useIngestJob.test.ts
web/src/hooks/useIngestJob.ts
web/src/hooks/useRallySelection.ts
web/src/hooks/useUpload.test.ts
web/src/hooks/useUpload.ts
web/src/i18n/catalog-invariants.test.ts
web/src/i18n/hygiene.test.ts
web/src/i18n/i18n.test.tsx
web/src/i18n/index.ts
web/src/index.css
web/src/lib/errorText.test.ts
web/src/lib/errorText.ts
web/src/lib/errors.test.ts
web/src/lib/errors.ts
web/src/lib/floor.test.ts
web/src/lib/floor.ts
web/src/lib/format.test.ts
web/src/lib/format.ts
```

```
web/src/lib/log.ts
web/src/lib/query.test.ts
web/src/lib/query.ts
web/src/lib/selection.test.ts
web/src/lib/selection.ts
web/src/main.tsx
web/src/test/setup.ts
web/src/vite-env.d.ts
web/tsconfig.json
web/vite.config.ts

=== cited src files exist? ===
EXISTS: web/src/api/client.ts
EXISTS: web/src/api/types.ts
EXISTS: web/src/lib/log.ts
EXISTS: web/src/App.tsx
```

7. user (2026-06-22T09:34:34.735Z)

```
=== cited docs exist? ===
EXISTS: docs/PER_RALLY_SELECTION.md
EXISTS: docs/LOCAL_APPLIANCE_ARCHITECTURE.md

=== web/package.json ===
{
  "name": "khelsutra-web",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "description": "KhelSutra Studio ? operator console SPA (drives the rally indexer engine).",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "typecheck": "tsc --noEmit",
    "test": "vitest run",
    "test:coverage": "vitest run --coverage"
  },
  "dependencies": {
    "@fontsource-variable/noto-sans-devanagari": "^5.2.8",
    "@fontsource-variable/noto-sans-kannada": "^5.2.9",
    "i18next": "^26.3.1",
    "i18next-browser-languagedetector": "^8.2.1",
    "i18next-icu": "^2.4.3",
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "react-i18next": "^17.0.8"
  },
  "devDependencies": {
    "@tailwindcss/vite": "^4.0.0",
    "@testing-library/dom": "^10.4.1",
    "@testing-library/react": "^16.3.2",
    "@types/node": "^26.0.0",
    "@types/react": "^19.0.0",
    "@types/react-dom": "^19.0.0",
    "@vitejs/plugin-react": "^4.3.4",
    "@vitest/coverage-v8": "^3.2.6",
    "jsdom": "^29.1.1",
    "tailwindcss": "^4.0.0",
    "typescript": "^5.7.0",
    "vite": "^6.0.0",
    "vitest": "^3.0.0"
  }
}
```

8. assistant (2026-06-22T09:34:37.895Z)

All cited paths exist and the package.json scripts match the doc's Dev/Checks sections. Now let me verify the load-bearing claims: the "Layout" section descriptions (especially `client.ts` exports and `App.tsx` being just "the console shell (screens land in M2+)"), and the `types.ts` "motion-only fabrications" claim.

9. user (2026-06-22T09:34:38.852Z)

```
1       import { useEffect, useMemo, useState } from "react";
2       import { useTranslation, Trans } from "react-i18next";
3       import { useRallySelection } from "./hooks/useRallySelection";
4       import { RallyGrid } from "./components/RallyGrid";
5       import { QueryBar } from "./components/QueryBar";
6       import { TapFilters } from "./components/TapFilters";
7       import { SelectionFooter } from "./components/SelectionFooter";
8       import { LanguageSwitcher } from "./components/LanguageSwitcher";
9       import { CompileResult, type BuildState } from "./components/CompileResult";
10      import { IngestPanel } from "./components/IngestPanel";
11      import { SetupWizard, isWizardDone } from "./components/SetupWizard";
12      import { runQuery, type ParsedQuery } from "./lib/query";
13      import { floorRisk } from "./lib/floor";
14      import { reelFilename } from "./lib/format";
15      import { classifyError } from "./lib/errors";
16      import { errorText } from "./lib/errorText";
17      import { log, newCorrelationId } from "./lib/log";
18      import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount }
from "./api/client";
19      import type { EngineConfig, RallySegment } from "./api/types";
20
21      /** One compact, structured description of how a query was understood (for the trace
log). */
22      function parseTrace(q: ParsedQuery) {
23        return {
24          raw: q.raw,
25          recognized: q.recognized,
26          filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
27          sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
28          limit: q.limit ?? null,
29          unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
30        };
31      }
32
33      // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
34      // contract; only honest fields are surfaced. A real session loads from POST /api/query.
35      const DEMO_RALLIES: RallySegment[] = [
36        { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
37        { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
38        { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
39        { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: {} },
40        { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
41        { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
42      ];
43
44      type LoadState =
45        | { status: "demo" }
46        | { status: "loading" }
47        | { status: "loaded"; count: number }
```

```
48        | { status: "error"; error: string };
49
50      export default function App() {
51        const { t, i18n } = useTranslation();
52        // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ? seeded from
the URL,
53        // but now also SETTABLE so the in-UI ingest panel (B-P2) can hand off a freshly-
processed video
54        // without a reload (it also rewrites ?video= so a refresh is stable).
55        const [videoId, setVideoId] = useState<string | null>(
56          () => new URLSearchParams(window.location.search).get("video"),
57        );
58
59        const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
60        const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
61        const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
62        const [reelName, setReelName] = useState("highlights");
63        const [build, setBuild] = useState<BuildState>({ status: "idle" });
64        // P2c: the first-run wizard shows once per browser, before the ingest panel, when
nothing's loaded.
65        const [wizardDone, setWizardDone] = useState(isWizardDone);
66
67        // Display order is separate from selection: a query can re-sort the cards while the
shared
68        // selection set (sel) stays the single source of truth for what lands in the reel.
69        const sel = useRallySelection(rallies, "all");
70
71        // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
72        useEffect(() => {
73          if (!videoId) return;
74          const cid = newCorrelationId();
75          log("info", "rallies.load_start", { cid, video_id: videoId });
76          // Show "loading" even when videoId was set dynamically by the ingest panel (initial
state was
77          // "demo" in that path) ? the banner must reflect the load, not stale demo copy.
78          setLoadState({ status: "loading" });
79          let alive = true;
80          listRallies(videoId, cid)
81            .then((rs) => {
82              if (!alive) return;
83              setRallies(rs);
84              setOrder(rs);
85              sel.apply(rs.map((r) => r.id));
86              setLoadState({ status: "loaded", count: rs.length });
87              log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
88            })
89            .catch((err) => {
90              if (!alive) return;
91              // Friendly, localized ? same shared mapping as build/ingest (A5): a load
failure on a slow or
92              // down engine reads "is it running?", not a raw "ApiError: Network error
calling POST ?".
93              const e = classifyError(err);
94              setLoadState({ status: "error", error: errorText(t, e) });
95              log("error", "rallies.load_failed", { cid, video_id: videoId, code: e.code,
detail: e.detail, error: String(err) });
96            });
97          return () => {
98            alive = false;
99          };
100         // videoId is stable for the page; sel.apply is a stable callback.
101         // eslint-disable-next-line react-hooks/exhaustive-deps
102       }, [videoId]);
103
```

```
      // Read the engine's stitching.min_shot_count so the footer can WARN before a selected rally is
      // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no warning.
      const [cfg, setCfg] = useState<EngineConfig | null>(null);
      useEffect(() => {
        let alive = true;
        getConfig()
          .then((c) => alive && setCfg(c))
          .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
        return () => {
          alive = false;
        };
      }, []);
      const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);

      // Stable chronological ordinal (by start_time) so re-sorting never renumbers a rally.
      const ordinalOf = useMemo(() => {
        const m = new Map<number, number>();
        [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) => m.set(r.id, i + 1));
        return (id: number) => m.get(id) ?? 0;
      }, [rallies]);

      // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the whole call.
      const onBuild = async () => {
        if (build.status === "building") return;
        const cid = newCorrelationId();
        const filename = reelFilename(reelName);
        const ids = [...sel.selected];
        if (!videoId) {
          log("info", "build.no_video", { cid, selected: ids.length });
          setBuild({
            status: "error",
            error: t("build.noVideo"),
          });
          return;
        }
        setBuild({ status: "building" });
        log("info", "build.start", {
          cid,
          video_id: videoId,
          filename,
          selected: ids.length,
          total_sec: Math.round(sel.totalSec),
          below_floor: warning?.atRisk.length ?? 0,
        });
        try {
          // Pass the active UI locale so the engine localizes the reel's branding watermark (LW1).
          const res = await compileReel(videoId, ids, filename, cid, i18n.resolvedLanguage ?? i18n.language);
          const url = highlightUrl(videoId, filename);
          setBuild({ status: "done", url, filename });
          log("info", "build.done", { cid, video_id: videoId, filename, filepath: res.filepath, url });
        } catch (err) {
          // Same friendly mapping the ingest flow uses: a bare 5xx / network throw reads "is the engine
          // running?", a real engine 4xx surfaces its own detail ? never a raw "HTTP 500" (A5).
          const status = err instanceof ApiError ? err.status : undefined;
          const e = classifyError(err);
          setBuild({ status: "error", error: errorText(t, e) });
```

```
160           log("error", "build.failed", { cid, video_id: videoId, filename, status, code:
e.code, detail: e.detail, error: String(err) });
161         }
162       };
163
164       // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
165       // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
166       // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
167       const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
168         const cid = newCorrelationId();
169         log("info", "query.parse", { cid, source, ...parseTrace(q) });
170
171         // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
172         // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
173         if (q.unavailable.length > 0) {
174           log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
175         }
176
177         if (!q.recognized) {
178           log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
179           return; // empty / unparseable ? leave the user's curation untouched
180         }
181
182         try {
183           const { ordered, selectedIds } = runQuery(rallies, q);
184           setOrder(ordered);
185           sel.apply(selectedIds);
186           log("info", "query.apply", {
187             cid,
188             source,
189             filters: q.filters.length,
190             sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
191             limit: q.limit ?? null,
192             total: rallies.length,
193             selected: selectedIds.length,
194             selected_ids: selectedIds,
195             dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
196           });
197           if (selectedIds.length === 0) {
198             // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
199             log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
200           }
201         } catch (err) {
202           // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
203           log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
204         }
205       };
206
207       // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
208       const onToggle = (id: number) => {
209         log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
210         sel.toggle(id);
211       };
212       const onBulk = (op: "select_all" | "clear" | "invert") => {
213         const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
```

```
rallies.length - sel.count;
214         log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
resultCount });
215         if (op === "select_all") sel.selectAll();
216         else if (op === "clear") sel.clearAll();
217         else sel.invert();
218       };
219
220       return (
221         <main className="min-h-screen bg-paper text-ink pb-24">
222           <header className="bg-ink text-white">
223             <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
py-4">
224               <span className="text-xl font-extrabold tracking-tight">
225                 Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
226               </span>
227               <LanguageSwitcher />
228             </div>
229           </header>
230
231           <section className="mx-auto max-w-5xl px-6 py-10">
232             <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
233               {t("app.badge")}
234             </span>
235             <h1 className="mt-4 text-3xl font-extrabold tracking-
tight">{t("app.title")}</h1>
236             <p className="mt-2 max-w-2xl text-ink/70">
237               <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
238             </p>
239
240             {/* P2c first-run wizard: before anything is loaded, walk a new user through AI-
key / compute /
241                 videos (from /api/capabilities). Shows once per browser; then the ingest
panel takes over. */}
242             {!videoId && !wizardDone && <SetupWizard onClose={() => setWizardDone(true)} />}
243
244             {/* Front-half ingest (B-P2): when no video is loaded, point the tool at a
bucket URI / host
245                 path ? process ? load. Once a video_id exists, the existing back-half takes
over. */}
246             {!videoId && wizardDone && (
247               <IngestPanel
248                 onLoaded={(id) => {
249                   setVideoId(id);
250                   window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
251                 }}
252               />
253             )}
254
255             <ModeBanner load={loadState} videoId={videoId} />
256
257             <div className="mt-6">
258               <QueryBar onApply={onApplyQuery} />
259               {/* A9: localized tap controls for users who can't type the English query
syntax. */}
260               <TapFilters onApply={onApplyQuery} />
261             </div>
262
263             <div className="mt-6 flex flex-wrap gap-2 text-sm">
264               <button type="button" onClick={() => onBulk("select_all")} className="inline-
flex min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.selectAll")}</button>
265               <button type="button" onClick={() => onBulk("clear")} className="inline-flex
```

```
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.clear")}</button>
266               <button type="button" onClick={() => onBulk("invert")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.invert")}</button>
267             </div>
268
269             <div className="mt-5">
270               <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
271             </div>
272
273             <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })} />
274           </section>
275
276           <SelectionFooter
277             count={sel.count}
278             totalSec={sel.totalSec}
279             reelName={reelName}
280             onReelNameChange={setReelName}
281             onBuild={onBuild}
282             building={build.status === "building"}
283             warning={warning}
284           />
285         </main>
286       );
287     }
288
289     function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
290       if (load.status === "demo") {
291         return (
292           <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
293             <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
294           </p>
295         );
296       }
297       if (load.status === "loading") {
298         return (
299           <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
300             <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code />
}} />
301           </p>
302         );
303       }
304       if (load.status === "error") {
305         return (
306           <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
text-red-800" data-testid="mode-banner">
307             <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
components={{ code: <code /> }} />
308           </p>
309         );
310       }
311       return (
312         <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
313           <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
<strong className="text-ink/70" /> }} />
314         </p>
315       );
316     }
317
```

10. user (2026-06-22T09:34:39.099Z)

```
1       // Typed client for the engine FastAPI. Default = SAME-ORIGIN relative `/api/*` (the
```

```
single-origin
2       // appliance reverse-proxies UI + engine behind one origin; in dev Vite proxies `/api` ?
:8000).
3       // Every call carries a correlation id (header + logs) so a journey is traceable end to
end.
4       import { log, newCorrelationId } from "../lib/log";
5       import type {
6         RallySegment,
7         QueryResponse,
8         CompileResponse,
9         EngineConfig,
10        EngineCapabilities,
11        ProcessResponse,
12        JobStatus,
13        UploadInitResponse,
14        UploadPartResponse,
15        UploadCompleteResponse,
16      } from "./types";
17
18      /** Resolve the engine API base. Default same-origin `"/api"`. For a SPLIT deploy ? SPA
on
19       *  Cloudflare Pages + the engine on a separate `api.*` cloudflared tunnel
(HOSTING_PLAN.md §2) ?
20       *  set `VITE_API_BASE` at build time to the engine's API base **including the `/api`
path**
21       *  (e.g. `https://api.khelsutra.guru/api`), and lock the engine's
`RALLY_CORS_ALLOW_ORIGINS` to the
22       *  Pages origin + put CF Access in front of both hostnames. A trailing slash is trimmed
so
23       *  `` `${base}/process` `` never double-slashes. */
24      export function apiBase(): string {
25        return (import.meta.env.VITE_API_BASE ?? "/api").replace(/\/+$/, "");
26      }
27
28      const BASE = apiBase();
29
30      export class ApiError extends Error {
31        readonly cid?: string;
32        readonly status?: number;
33        readonly body?: string;
34        constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
35          super(message);
36          this.name = "ApiError";
37          this.cid = opts.cid;
38          this.status = opts.status;
39          this.body = opts.body;
40          if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
41        }
42      }
43
44      interface RequestOpts {
45        method?: string;
46        body?: string;
47        headers?: Record<string, string>;
48        /** Correlation id to thread through this call; auto-generated if absent. */
49        cid?: string;
50      }
51
52      async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
53        const cid = opts.cid ?? newCorrelationId();
54        const url = `${BASE}${path}`;
55        const method = opts.method ?? "GET";
56        log("info", "api.request", { cid, method, url });
57
```

```
58        let res: Response;
59        try {
60          res = await fetch(url, {
61            method,
62            body: opts.body,
63            headers: { "content-type": "application/json", "x-correlation-id": cid,
...(opts.headers ?? {}) },
64          });
65        } catch (err) {
66          // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
67          log("error", "api.network_error", { cid, method, url, error: String(err) });
68          throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
69        }
70
71        if (!res.ok) {
72          const body = await res.text().catch(() => "");
73          log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
74          throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
75        }
76
77        const data = (await res.json()) as T;
78        log("info", "api.response", { cid, method, url, status: res.status });
79        return data;
80      }
81
82      /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
for
83       *  processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-
fails 400
84       *  (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
85      export async function processVideo(videoPath: string, cid?: string, locale?: string):
Promise<ProcessResponse> {
86        // `locale` (the UI language) is recorded for PMF analytics (which markets use the
tool). Optional
87        // + omitted when absent (JSON.stringify drops undefined); the engine ignores it until
wired.
88        return request<ProcessResponse>("/process", {
89          method: "POST",
90          body: JSON.stringify({ video_path: videoPath, locale }),
91          cid,
92        });
93      }
94
95      /** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
96      export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
97        return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});
98      }
99
100     /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
101     export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
102       const data = await request<QueryResponse>("/query", {
103         method: "POST",
104         body: JSON.stringify({ video_id: videoId }),
105         cid,
106       });
107       return data.play_segments ?? [];
108     }
109
110     /** Stitch selected rallies into one reel. POST /api/compile {video_id, segment_ids,
```

```
output_filename} [main.py:342]. */
    export async function compileReel(
      videoId: string,
      segmentIds: number[],
      outputFilename: string,
      cid?: string,
      locale?: string,
    ): Promise<CompileResponse> {
      // `locale` (the UI language) localizes the reel's branding watermark engine-side.
Optional +
      // omitted when absent; the engine falls back to the default watermark for an
unknown/absent locale.
      return request<CompileResponse>("/compile", {
        method: "POST",
        body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename, locale }),
        cid,
      });
    }

    /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
    export function highlightUrl(videoId: string, filename: string): string {
      return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
    }

    /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
    export async function getConfig(cid?: string): Promise<EngineConfig> {
      return request<EngineConfig>("/config", { method: "GET", cid });
    }

    /** What THIS box can do ? drives the first-run wizard. GET /api/capabilities
(PACKAGING_PLAN P2c). */
    export async function getCapabilities(cid?: string): Promise<EngineCapabilities> {
      return request<EngineCapabilities>("/capabilities", { method: "GET", cid });
    }

    /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
    export function minShotCount(cfg: EngineConfig): number | undefined {
      return cfg.stitching?.min_shot_count;
    }

    // ?? chunked upload (backend/api/uploads.py) ????????????????????????????????
    // A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge
proxies cap
    // bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.

    /** Start a chunked upload. 400 = bad extension; 413 = over the size limit
[uploads.py:64]. */
    export async function uploadInit(filename: string, totalSizeBytes: number, cid?:
string): Promise<UploadInitResponse> {
      return request<UploadInitResponse>("/upload/init", {
        method: "POST",
        body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
        cid,
      });
    }

    /** Append ONE part (raw bytes ? not JSON). Parts are strictly sequential 0,1,2,?
[uploads.py:89]. */
    export async function uploadPart(
      uploadId: string,
```

```
163       index: number,
164       chunk: Blob,
165       cid?: string,
166       signal?: AbortSignal,
167     ): Promise<UploadPartResponse> {
168       const c = cid ?? newCorrelationId();
169       const url = `${BASE}/upload/${encodeURIComponent(uploadId)}/part/${index}`;
170       log("info", "api.request", { cid: c, method: "PUT", url });
171       let res: Response;
172       try {
173         res = await fetch(url, {
174           method: "PUT",
175           body: chunk,
176           headers: { "content-type": "application/octet-stream", "x-correlation-id": c },
177           signal,
178         });
179       } catch (err) {
180         log("error", "api.network_error", { cid: c, method: "PUT", url, error: String(err)
});
181         throw new ApiError(`Network error calling PUT ${url}`, { cid: c, cause: err });
182       }
183       if (!res.ok) {
184         const body = await res.text().catch(() => "");
185         log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status,
body: body.slice(0, 500) });
186         throw new ApiError(`PUT ${url} failed: ${res.status}`, { cid: c, status: res.status,
body });
187       }
188       log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });
189       return (await res.json()) as UploadPartResponse;
190     }
191
192     /** Finalize: returns {path, video_id} ? feed `path` to processVideo() [uploads.py:123].
*/
193     export async function uploadComplete(uploadId: string, totalParts: number, cid?:
string): Promise<UploadCompleteResponse> {
194       return
request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
195         method: "POST",
196         body: JSON.stringify({ total_parts: totalParts }),
197         cid,
198       });
199     }
200
201     /** Abort an in-flight upload (best-effort cleanup of the server-side partial)
[uploads.py:147]. */
202     export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
203       await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE",
cid });
204     }
205
206     export interface UploadProgress {
207       sentBytes: number;
208       totalBytes: number;
209     }
210
211     /** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ?
complete.
212      *  On ANY failure or abort it best-effort DELETEs the server-side partial, then
rethrows so the
213      *  caller can classify it (the engine's 400/413 detail survives). Returns {path,
video_id}.
214      *  Note: a per-part 413 (over the per-part limit) aborts the upload server-side ?
recovery is a
215      *  fresh init (re-pick the file), which the UI's retry does. */
```

```
216     export async function uploadFile(
217       file: File,
218       opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void }
= {},
219     ): Promise<UploadCompleteResponse> {
220       const cid = opts.cid ?? newCorrelationId();
221       const init = await uploadInit(file.name, file.size, cid);
222       // Honor the engine's max_part_mb, but never let a missing/NaN value collapse the math
to NaN (which
223       // would silently upload zero parts) ? fall back to the engine's 95 MB default.
224       const partMb = Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ?
init.max_part_mb : 95;
225       const partBytes = partMb * 1024 * 1024;
226       const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
227       log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:
totalParts, part_mb: init.max_part_mb });
228       let sent = 0;
229       try {
230         for (let i = 0; i < totalParts; i++) {
231           if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
232           const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
233           await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
234           sent += chunk.size;
235           opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
236         }
237         const done = await uploadComplete(init.upload_id, totalParts, cid);
238         log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:
done.size_bytes });
239         return done;
240       } catch (err) {
241         log("warn", "upload.aborting_partial", { cid, upload_id: init.upload_id, error:
String(err) });
242         await uploadAbort(init.upload_id, cid).catch(() => {});
243         throw err;
244       }
245     }
246
```

11. assistant (2026-06-22T09:34:45.381Z)

This is a significant finding. The README's "Layout" section is materially stale. Let me verify the remaining specific claims:

1. `client.ts` exports ? doc says `listRallies / compileReel / highlightUrl / getConfig`. Actual exports include many more (processVideo, getJob, getCapabilities, minShotCount, uploadInit/Part/Complete/Abort/File). So the doc's list is incomplete but each named one exists.
2. `App.tsx` ? doc says "the console shell (screens land in M2+)" ? but App.tsx is now a full screen with rally grid, query bar, selection footer, ingest panel, setup wizard, etc. The "screens land in M2+" claim is clearly stale.
3. `types.ts` "motion-only fabrications" claim ? let me verify.

Let me check `types.ts` and the engine dev command claim, plus confirm the dev port/proxy config.

12. user (2026-06-22T09:34:46.335Z)

```
1       // Types mirror the engine's verified contract (badminton-highlight-indexer).
2       // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3       // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5       export interface RallyMetadata {
6         /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7         shot_count?: number;
8         /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
```

```
9         shot_count_confidence?: string;
10        detector?: string;
11        [key: string]: unknown;
12      }
13
14      export interface RallySegment {
15        /** The integer the UI selects and passes to /api/compile as segment_ids. */
16        id: number;
17        start_time: number;
18        end_time: number;
19        duration: number;
20        /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21        server: string | null;
22        /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23        receiver: string | null;
24        metadata: RallyMetadata;
25        /** motion-only fabricated sub-events ? DO NOT surface. */
26        events?: unknown[];
27      }
28
29      export interface QueryResponse {
30        play_segments: RallySegment[];
31      }
32
33      export interface CompileResponse {
34        status: string;
35        /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
36        filepath: string;
37      }
38
39      export interface EngineConfig {
40        stitching?: {
41          /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
*/
42          min_shot_count?: number;
43          [key: string]: unknown;
44        };
45        [key: string]: unknown;
46      }
47
48      /** GET /api/capabilities [backend/main.py] ? an HONEST probe of what THIS box can do,
rendered by
49       *  the first-run wizard (PACKAGING_PLAN.md P2c). Presence-bools only (never the Gemini
key value).
50       *  Every field optional/defensive: a missing field degrades the wizard gracefully. */
51      export interface EngineCapabilities {
52        api_version?: number;
53        engine_version?: string;
54        segmenters?: string[];
55        default_segmenter?: string;
56        ffmpeg?: { ffmpeg?: boolean; ffprobe?: boolean; ffmpeg_path?: string; ffprobe_path?:
string };
57        gpu?: { nvidia_smi_present?: boolean };
58        /** Whether a Gemini key is PRESENT (never its value) ? drives the AI-highlights step.
*/
59        gemini_key_present?: boolean;
60        wasb_weights_present?: boolean;
61        deployment?: { profile?: string; compute_target?: string };
62        [key: string]: unknown;
63      }
64
65      /** POST /api/process ? 202 [backend/main.py:260]. The job is queued; poll GET
/api/jobs/{job_id}. */
66      export interface ProcessResponse {
```

```
67        job_id: string;
68        /** Always "queued" at submit. */
69        status: string;
70        /** Relative poll path, e.g. "/api/jobs/<id>". */
71        poll: string;
72      }
73
74      /** Chunked upload (backend/api/uploads.py). The browser uploads a local file in
sequential parts
75       *  ? max_part_mb, then `complete` returns a server-host `path` to feed straight into
POST /api/process. */
76      export interface UploadInitResponse {
77        upload_id: string;
78        /** Per-part ceiling in MB ? slice the file into parts this big. */
79        max_part_mb: number;
80        next_part: string;
81      }
82      export interface UploadPartResponse {
83        upload_id: string;
84        received_parts: number;
85        received_bytes: number;
86      }
87      export interface UploadCompleteResponse {
88        /** Absolute path on the engine host ? pass as video_path to POST /api/process. */
89        path: string;
90        video_id: string;
91        size_bytes: number;
92      }
93
94      /** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
95       *  `status` = the EXECUTION state; `result.status` = the pipeline VERDICT
(success|failed).
96       *  A `done` job whose `result.status === "failed"` (or with no video_id) is a success-
shaped
97       *  FAILURE ? the UI must not treat it as a load. */
98      export interface JobStatus {
99        job_id: string;
100       status: "queued" | "running" | "done" | "failed";
101       /** Free-text coarse phase ("hashing", "segmenting (gemini)", "finished") ? NOT a
percentage. */
102       progress?: string | null;
103       /** The canonical video_id, surfaced top-level on success (also in result.video_id).
*/
104       video_id?: string | null;
105       /** Execution error (worker crash / unexpected) when status === "failed". */
106       error?: string | null;
107       result?: { status?: string; video_id?: string; message?: string } | null;
108     }
109
```

13. user (2026-06-22T09:34:46.809Z)

```
1       import { defineConfig } from "vitest/config";
2       import react from "@vitejs/plugin-react";
3       import tailwindcss from "@tailwindcss/vite";
4
5       // The SPA makes SAME-ORIGIN relative `/api/*` calls. In the appliance (prod) a reverse
6       // proxy serves the SPA and proxies `/api` to the engine on 127.0.0.1:8000
7       // (docs/LOCAL_APPLIANCE_ARCHITECTURE.md). In dev, Vite proxies `/api` to the local
engine
8       // (`python -m backend.main --server`, :8000). Override with VITE_ENGINE_ORIGIN if
needed.
9       export default defineConfig({
10        plugins: [react(), tailwindcss()],
11        server: {
```

```
12          proxy: {
13            "/api": { target: "http://localhost:8000", changeOrigin: true },
14          },
15        },
16        test: {
17          // Default env is node (the pure lib/parser tests); the i18n render test opts into
jsdom via a
18          // `// @vitest-environment jsdom` docblock. Match .test.tsx too so render tests are
discovered.
19          environment: "node",
20          include: ["src/**/*.test.{ts,tsx}"],
21          // DOM-env localStorage polyfill (jsdom under Node 26 ships none); node-env files
no-op.
22          setupFiles: ["src/test/setup.ts"],
23          // Coverage floor (CI runs `npm run test:coverage`): a merge can't DROP coverage
below these
24          // thresholds, so untested new code is caught. `all` counts every source file (not
only the ones a
25          // test imported), so deleting a test surfaces here too. Excludes are non-logic
files: the app
26          // entry/bootstrap, type-only declarations, and the test harness itself.
27          coverage: {
28            provider: "v8",
29            reporter: ["text", "text-summary"],
30            all: true,
31            include: ["src/**/*.{ts,tsx}"],
32            exclude: [
33              "src/**/*.test.{ts,tsx}", // the tests
34              "src/test/**", // jsdom localStorage shim (test harness, not shipped logic)
35              "src/main.tsx", // app entry: mounts <App/> + bundles fonts ? nothing to unit-
test
36              "src/api/types.ts", // type-only declarations (no runtime code)
37              "**/*.d.ts",
38            ],
39            // Floor set just below current (stmts/lines 93.3, branches 88.1, funcs 85.9) ? a
real ratchet
40            // that fails CI on a meaningful drop, with a little slack so it doesn't flap.
Ratchet upward as
41            // coverage grows. (functions stays a touch looser: App.tsx is the engine-
orchestrating shell,
42            // costly to exercise fully in unit tests.)
43            thresholds: {
44              statements: 92,
45              branches: 87,
46              functions: 84,
47              lines: 92,
48            },
49          },
50        },
51      });
52
```

14. user (2026-06-22T09:34:47.269Z)

```
1       // Structured client logger ? the observability seed for the operator console
2       // (docs/LOCAL_APPLIANCE_ARCHITECTURE.md §5). Every API call carries a CORRELATION ID so
a
3       // single user journey is traceable across the UI ? FastAPI ? worker. Logs are JSON-
shaped
4       // so they aggregate cleanly off a transient host. Keep this dependency-free.
5
6       export type LogLevel = "debug" | "info" | "warn" | "error";
7
8       /** A short, unique id to correlate all log lines (and the x-correlation-id header) for
one action. */
```

```
9       export function newCorrelationId(): string {
10        const c = (globalThis as { crypto?: Crypto }).crypto;
11        if (c && typeof c.randomUUID === "function") return c.randomUUID();
12        // Fallback for environments without crypto.randomUUID.
13        return `cid_${Date.now().toString(36)}_${Math.floor(Math.random() *
1e9).toString(36)}`;
14      }
15
16      export function log(level: LogLevel, event: string, fields: Record<string, unknown> =
{}): void {
17        const line = JSON.stringify({ ts: new Date().toISOString(), level, event, ...fields
});
18        if (level === "error") console.error(line);
19        else if (level === "warn") console.warn(line);
20        else console.log(line);
21      }
22
```

15. assistant (2026-06-22T09:34:53.242Z)

Now I have strong evidence. Let me verify the remaining claims:

1. The `types.ts` claim about "server/receiver/events are motion-only fabrications and must not be surfaced" ? CONFIRMED (lines 20-26).
2. The `log.ts` claim ? CONFIRMED.
3. The dev port 5173 and proxy `/api` ? `:8000` ? CONFIRMED in vite.config.ts.
4. The engine dev command. The README says `python -m backend.main --server`, vite.config.ts agrees. Let me verify against the actual badminton-highlight-indexer engine to confirm that's the real invocation (the doc also says `python -m backend.main --server` vs CLAUDE.md mentions `backend.main --server`). Let me also check the "M2+" framing and whether the README has been touched recently.

16. user (2026-06-22T09:34:55.684Z)

```
=== git log for web/README.md ===
aac7e0c feat(web): per-rally MVP P1 ? SPA scaffold + typed engine API client (#14)
a6f616c init: KhelSutra Guru product repo

=== last commit touching App.tsx ===
7ae9cee feat(web): P2c first-run setup wizard (3 steps from /api/capabilities, 6 languages)
(#88)
c1848df feat(web): LW2 ? send the UI locale to /api/compile (watermark) + /api/process
(analytics) (#73)
46e0244 feat(web): A9 ? localized tap-only filter controls (#68)
501c014 feat(web): A5 ? friendly engine-error mapping across build + load (shared errors.*)
(#62)
95195a5 feat(web): A4 ? mobile touch targets (?44px) + visible tooltip text (#61)
```

17. user (2026-06-22T09:34:57.058Z)

```
=== engine --server flag? ===
867:    parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
871:    parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
backend/main.py:867:    parser.add_argument("--server", action="store_true", help="Run the
FastAPI local API server")
```

18. assistant (2026-06-22T09:35:05.204Z)

This confirms the picture definitively. The README was last touched at PR #14 (the P1 scaffold) and has NOT been updated since, despite App.tsx having grown through PRs #61, #62, #68, #73, #88, etc. ? the full per-rally selection UI, ingest panel, setup wizard, and 6-language i18n all shipped after the README froze.

Key findings:
- The engine `--server` flag is real (backend/main.py:867) and port defaults to 8000 ? the Dev/proxy claims are CORRECT.
- `App.tsx` is NO LONGER "the console shell (screens land in M2+)" ? it's a fully-built screen. This is the most misleading claim.
- The `client.ts` export list is incomplete (omits processVideo, getJob, getCapabilities, minShotCount, the whole upload family) but the four named exports do exist.
- The "Layout" section omits the entire `components/`, `hooks/`, `i18n/` trees and most of `lib/`.
- The `types.ts` and `log.ts` descriptions are still accurate.

Let me confirm one nuance: the README says "every call carries a correlation id" for client.ts ? confirmed in client.ts. And verify the i18n/6-languages context to assess whether the README's omission of it is a staleness finding. Let me quickly confirm the locale count.

19. user (2026-06-22T09:35:07.276Z)

```
=== locales in i18n ===
"da"
"en"
"es"
"hi"
"id"
"kn"

=== component count ===
9
```

20. assistant (2026-06-22T09:35:11.620Z)

I have all the evidence I need. The README froze at the P1 scaffold (#14) and the SPA has since grown a full UI (per-rally selection screen, ingest panel, P2c setup wizard, 6-language i18n, 9 components, hooks, lib utils). The Dev/Checks sections and the `types.ts`/`log.ts`/proxy descriptions remain accurate; the "Layout" section and especially the "App.tsx = console shell (screens land in M2+)" claim are materially stale.

21. user (2026-06-22T09:35:38.311Z)

Structured output provided successfully

22. assistant (2026-06-22T09:35:52.711Z)

Audit complete. `web/README.md` is mostly fresh but has one high-severity stale claim and a stale Layout section.

The doc froze at PR #14 (its only content commit besides repo init) while the SPA grew through ~PR #88. The most misleading line is `web/README.md:30` ? "`src/App.tsx` ? the console shell (screens land in M2+)" ? which is now wrong: `web/src/App.tsx` is a fully-built per-rally selection screen (RallyGrid, QueryBar, TapFilters, SelectionFooter, IngestPanel, the P2c SetupWizard, 6-language i18n). The "Layout" section (`web/README.md:24-30`) also omits the entire `components/`, `hooks/`, `i18n/` trees and most of `lib/`.

Everything else checks out against code: the Dev/Checks commands match `web/package.json`, the `/api ? :8000` proxy matches `web/vite.config.ts:11-14`, `python -m backend.main --server` is real (`backend/main.py:867` in the sibling engine repo), and the `types.ts`/`log.ts` descriptions are accurate. Not an archive candidate ? it's the live, correct dev/CI entry point that just needs a Layout refresh.